

RQL: A Portable Intermediate Representation for Retrieval Plans in RAG Systems

Incremental research draft — not a camera-ready submission

Vijay Kumar

Research draft, Europe/Dublin

`vijaykumarjob0701/rql-rag-query-language`

16 September 2026

Draft status. Structured for growth into a citable article/thesis. Claims are labelled [ESTABLISHED], [HYPOTHESIS], or [PROVISIONAL]. No fabricated experimental results. Not claimed ready for NeurIPS, VLDB, or arXiv acceptance.

Abstract

Retrieval-augmented generation (RAG) systems compose embedding search, lexical retrieval, metadata and ACL filters, hybrid fusion, reranking, and sometimes graph or multi-hop expansion. Today those stages are usually glued together in application code against vendor-specific vector-database APIs. This draft proposes **RQL (Retrieval Query Language)** not merely as “SQL for vectors”, but as a **portable intermediate representation for retrieval plans**: a small algebra over scored evidence, plus cost-based physical planning under recall, latency, and ACL constraints, compiling through adapters to heterogeneous backends.

We separate [ESTABLISHED] survey material from [HYPOTHESIS] original design and from [PROVISIONAL] literature pending multimodal re-read. A careful multimodal pass over ACORN [Patel et al., 2024] grounds filtered-ANN physical strategies. We define an evaluation *plan* and reproducibility layout; we do **not** report unreproduced benchmark scores.

1 Introduction

1.1 Problem

Modern RAG stacks must retrieve *evidence*—chunks, passages, entities, paths—under mixed modalities: dense vectors, sparse/BM25 channels, structured predicates (including ACLs), and increasingly late-interaction or graph expansion [Lewis et al., 2020]. Vector databases expose powerful but **non-portable** interfaces (JSON filters, GraphQL operators, SQL extensions, Redis commands, Cypher search). As a result, accuracy, latency, and reliability become properties of brittle Python glue rather than of inspectable plans.

1.2 Motivation (accuracy, latency, reliability)

Accuracy. Hybrid fusion (e.g. reciprocal rank fusion [Cormack et al., 2009]), over-fetch+rerank, and multi-query rewriting improve recall only when stages are explicit and tunable.

Latency. Filter placement (pre vs. post vs. subgraph traversal) and ANN parameters dominate end-to-end cost [Patel et al., 2024].

Reliability. Silent empty results, client-side ACL filtering, and irreproducible `efSearch` folklore are operational failures; plans with `EXPLAIN` and mandatory predicate pushdown targets are the remedy [HYPOTHESIS].

1.3 Thesis statement

RQL is a portable IR for retrieval plans—an algebra over scored multisets of evidence together with a Cascades/Calcite-style physical planner that chooses filtered-ANN and fusion strategies under explicit recall/latency/ACL contracts, then compiles via polystore-like shims to concrete engines. The textual DSL is a frontend, not the contribution’s centre of mass [HYPOTHESIS].

1.4 Contributions (draft)

1. A clear problem formulation for portable retrieval plans (Section 4).
2. A hypothesized retrieval algebra and physical operator set (Sections 5–6) [HYPOTHESIS].
3. Compilation/adaptor doctrine inspired by Substrait and polystores (Section 7) [HYPOTHESIS].
4. An evaluation and reproducibility *plan* without fabricated metrics (Section 9).
5. Careful related-work positioning, with ACORN multimodally re-read (Section 3).

2 Background

2.1 Approximate nearest neighbour search

Graph indices such as HNSW [Malkov and Yashunin, 2018] and disk-oriented graphs (DiskANN line) underpin production vector search. Search quality/latency is controlled by parameters (e.g. candidate list size), yielding recall–throughput curves rather than exact SQL cardinalities [ESTABLISHED].

2.2 Hybrid vector + predicate search

Applications need ANN jointly with structured predicates. Common strategies include **pre-filtering**, **post-filtering**, specialised low-cardinality equality indices, and **predicate-subgraph** traversal as in ACORN [Patel et al., 2024] [ESTABLISHED] (see Section 3).

2.3 Hybrid lexical–dense fusion

Reciprocal Rank Fusion (RRF) combines ranked lists without score calibration [Cormack et al., 2009] [ESTABLISHED]. Production engines also expose weighted linear fusion and learned rankers; choice is workload-dependent [PROVISIONAL].

2.4 Late interaction

ColBERT-style late interaction [Khattab and Zaharia, 2020] and subsequent engines/rewrites (e.g. MU-VERA [Dhulipala et al., 2024]) show that multi-vector retrieval admits planner rewrites [PROVISIONAL] (pending dedicated multimodal reads).

2.5 RAG as a staged pipeline

A typical RAG retrieve path is: embed → search channels → filter → fuse → (rerank) → expand/traverse → generate. RQL aims to name these stages as plan operators rather than ad-hoc functions [HYPOTHESIS].

3 Related work

3.1 Vector database interfaces

Commercial and OSS systems expose vendor-specific query surfaces (metadata filters, hybrid search APIs, SQL extensions such as pgvector). Survey details and URLs live in the companion research package; none is a cross-vendor ANSI-SQL analogue for multi-stage RAG plans [ESTABLISHED] (landscape notes).

3.2 Filtered ANN / FANNS

ACORN (multimodal re-read). Patel et al. [Patel et al., 2024] argue that pre- and post-filtering are brittle under selectivity and query–predicate correlation, and that specialised equality indices constrain predicate cardinality. They propose denser, predicate-agnostic HNSW variants (ACORN- γ , ACORN-1) that *search the predicate subgraph* to approximate an oracle HNSW over surviving vectors. Our Pass 1–5 notes (journal 0006; OKF bundle under `knowledge/reads/acorn-2403.04871/`) treat author QPS claims as **not reproduced here** [ESTABLISHED] for the problem taxonomy; [PROVISIONAL] for unreproduced numeric speedups.

Filtered-DiskANN and others. Filtered-DiskANN [Gollapudi et al., 2023] and related specialised structures target constrained equality predicates [PROVISIONAL] (queued for multimodal read). VBASE [Zhang et al., 2023] frames vector similarity as composable with relational execution via relaxed monotonicity [PROVISIONAL] (journal stub 0007 only).

3.3 IR query languages

Classical IR languages (Indri, Galago, CQL/SRU) treat ranking as an operator algebra [met, 2004] [PROVISIONAL] (URL-verified references; deep re-read pending). RQL steals composability, not surface syntax fidelity [HYPOTHESIS].

3.4 Optimisers, portable plans, polystores

Apache Calcite [Begoli et al., 2018] and Substrait [sub, 2024] motivate logical/physical separation and portable plan IR. BigDAWG-style polystores [Duggan et al., 2015] motivate islands + shims [PROVISIONAL]. BlinkDB [Agarwal et al., 2013] motivates explicit approximate budgets [PROVISIONAL].

3.5 Knowledge packaging (methodology, not runtime)

Google’s Open Knowledge Format (OKF) packages knowledge as markdown concepts with YAML frontmatter and graph-shaped links [okf, 2025]. We use OKF-*shaped* bundles for research notes. We do **not** claim OKF is part of the RQL execution runtime [ESTABLISHED].

3.6 What RQL is not

RQL is not “another vendor filter DSL”, not a guarantee of identical recall across HNSW libraries, and not a claim that SQL alone solves multi-stage RAG [HYPOTHESIS].

4 Problem formulation

4.1 Setting

Let collections store entities with payloads, optional graph links, and one or more retrieval channels (dense vectors, lexical fields, multi-vectors). A **retrieval plan** consumes a user/agent query (text and/or

vectors), mandatory security predicates, and optional budgets, and returns a ranked evidence multiset.

4.2 Requirements

1. **Portability:** one logical plan maps to multiple backends via adapters.
2. **Filter correctness:** ACL/tenant predicates are hard constraints (fail closed).
3. **Explicit approximation:** recall/latency targets are first-class, not hidden knobs.
4. **Composability:** fusion, rewrite, expand, traverse, and similarity joins are plan nodes.
5. **Observability:** logical/physical EXPLAIN with provenance for evaluation harnesses.

4.3 Non-goals (this draft)

Identical numeric recall across ANN implementations; full DDL standardisation; replacing embedding model APIs; unreproduced SOTA claims.

5 RQL algebra (original sketch)

All operators in this section are [HYPOTHESIS] unless marked otherwise.

5.1 Domain

An **evidence relation** E is a multiset of tuples

$$(id, payload, score, channel, provenance, \dots).$$

Scores are channel-local until a fuse operator.

5.2 Logical operators (kernel)

Operator	Informal type	Notes
Search_{dense}	$\rightarrow E$	ANN leaf
Search_{bm25}	$\rightarrow E$	Lexical leaf
Search_{late}	$\rightarrow E$	Late interaction leaf
$\text{Filter}(P)$	$E \rightarrow E$	Predicates / ACL
Union	$E \times E \rightarrow E$	Multi-query
$\text{Fuse}_{rrf} / \text{Fuse}_{linear}$	$E^* \rightarrow E$	RRF [ESTABLISHED] as prior art; naming in RQL [HYPOTHESIS]
Diversify_{mmr}	$E \rightarrow E$	Diversity
Rerank	$E \rightarrow E$	Cross-encoder etc.
Expand	$E \rightarrow E$	Parent/window
Rewrite	$q \rightarrow q^*$	HyDE / multi-query
Traverse	$E \rightarrow E$	Bounded graph hop
VSimJoin	$E \times E \rightarrow E$	Similarity join

5.3 Semantics sketch

Channels remain separate until fuse. Mandatory predicates in $P_{acl} \subseteq P$ must be enforced by physical plans that cannot strip them client-side [HYPOTHESIS]. Rewrite operators carry explicit token/latency cost attributes for the optimiser.

5.4 Delineation from survey material

RRF [Cormack et al., 2009], HNSW [Malkov and Yashunin, 2018], and ACORN’s predicate-subgraph idea [Patel et al., 2024] are prior art. **Original to this draft:** packaging them as a single portable retrieval IR with explicit physical filter modes and budget contracts aimed at RAG reliability [HYPOTHESIS].

6 Physical planning (original sketch)

This section is [HYPOTHESIS].

6.1 Physical filter modes

Inspired by FANNS literature and ACORN’s taxonomy [Patel et al., 2024]:

$\text{FilterExec} \in \{\text{PRE}, \text{POST}, \text{ITERATIVE}, \text{SUBGRAPH}, \text{SPECIALIZED}, \text{AUTO}\}.$

- PRE: restrict candidates then exact/ANN on survivors.
- POST: ANN then predicate; often needs over-fetch.
- SUBGRAPH: ACORN-like traversal when the backend capability exists.
- SPECIALIZED: equality-constrained indices (e.g. Filtered-DiskANN class) when applicable [PROVISIONAL].
- ITERATIVE: iterator/scan integration (VBASE-class) [PROVISIONAL].
- AUTO: choose using stats; **never** violate ACL hard constraints.

6.2 Cost vector

Unlike classical selectivity-only costing, plans expose a vector:

1. Recall proxy (candidate depth, filter empty-result risk),
2. Latency (p50/p95 targets),
3. ACL safety (boolean hard constraint),
4. Token/dollar budget (embeddings, rerankers, rewrites).

BlinkDB-style explicit budgets [Agarwal et al., 2013] motivate surface options such as `RECALL TARGET / LATENCY` [HYPOTHESIS].

6.3 Fusion and rewrite rules

Examples [HYPOTHESIS]: independent channel searches $\rightarrow$ parallel schedule; Search_{late} without multi-vector support $\rightarrow$ optional FDE-style rewrite if available [Dhulipala et al., 2024]; low selectivity + POST $\rightarrow$ rewrite toward ITERATIVE/SUBGRAPH/over-fetch.

7 Compilation and adapters

[HYPOTHESIS] doctrine; Subtrait/polystore inspiration is [PROVISIONAL]/[ESTABLISHED] as prior systems.

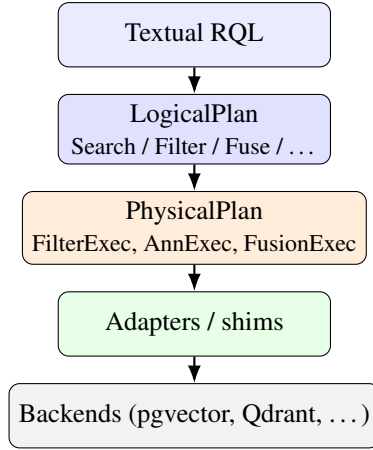


Figure 1: Hypothesized RQL compilation stack: text $\rightarrow$ logical plan $\rightarrow$ physical plan $\rightarrow$ adapters.

7.1 Layers

1. Parse textual RQL to `LogicalPlan`.
2. Optimise to `PhysicalPlan` with capability negotiation.
3. Emit backend calls through adapters (pgvector SQL, Qdrant query API, Pinecone, Neo4j, ...).

7.2 Capability negotiation

Profiles declare required features (hybrid+RRF, graph hop, late interaction). Missing capabilities yield either a safe rewrite, a shim CAST (e.g. client-side fuse—explicitly marked expensive/unsafe for ACL), or a hard error [HYPOTHESIS].

7.3 Substrait adjacency

Substrait [sub, 2024] motivates a binary/portable plan encoding across engines. RQL may eventually emit a retrieval-oriented dialect or embed custom extensions; this draft does not claim a finished Substrait specification [HYPOTHESIS].

8 Applications to RAG patterns

Patterns below map to example programs in the companion repo (`examples/*.rql`).

- Dense + ACL metadata filter (reliability-first pushdown).
- Hybrid BM25+dense with RRF and optional cross-encoder rerank [Cormack et al., 2009].
- Multi-query / HyDE-style rewrite as plan nodes (costed) [PROVISIONAL].
- Parent expansion and bounded multi-hop graph retrieval [PROVISIONAL].
- Late-interaction search with optional planner rewrite [PROVISIONAL].

RQL’s value is making these patterns *comparable* under EXPLAIN and shared eval harnesses [HYPOTHESIS].

9 Evaluation plan (no fabricated results)

*This section specifies **what we will measure**, not results we have not run.*

9.1 Datasets (planned)

- Retrieval: BEIR/MS MARCO-style collections as available under license; synthetic metadata predicates for filter selectivity sweeps.
- Filtered-ANN microbenchmarks aligned with published FANNS setups (e.g. SIFT-scale vectors with attribute predicates), following protocols from prior work such as ACORN [Patel et al., 2024]—**re-run**, do not copy tables.

HUMAN_TODO: dataset download, licenses, and digests — see `experiments/HUMAN_TODO.md` and `experiments/protocols/02-datasets.md`.

9.2 Metrics

Recall@ k , nDCG@ k , latency p50/p95, throughput (QPS) where applicable, **plan stability** (same RQL → same logical plan hash), and ACL violation count (must be zero).

9.3 Baselines

1. Hand-written SDK glue (LangChain/LlamaIndex-style scripts).
2. Vendor filter+hybrid APIs directly.
3. pgvector SQL hybrid patterns.
4. Fixed `FILTER_MODE` ablations vs AUTO (once implemented).

9.4 Ablation matrix (planned)

Channels $\in \{\text{dense, bm25, both}\}$; fuse $\in \{\text{RRF, linear}\}$; filter mode $\in \{\text{PRE, POST, AUTO}\}$; rerank on/off; rewrite on/off.

9.5 What exists today (agent-run)

- Planned-run printer: `experiments/harness/print_planned_runs.py` (emits `score=NOT_RUN` only).
- Deterministic RRF unit test on fixed lists: `experiments/harness/test_rrf_fusion.py` → `experiments/results/rrf_unit_test.txt`.

No end-to-end ANN, RAG, or human-judgment metrics are claimed.

9.6 What requires a human (Vijay)

GPU/CPU FANNS microbench, production/docker vector DB smokes, paid APIs, large datasets, and relevance judgments are listed as **HUMAN_TODO** items with exact protocols and expected in-repo artifact paths in `experiments/HUMAN_TODO.md`. Those results should be pushed to this same GitHub repository when available.

10 Threats to validity

- **Literature coverage:** only ACORN has completed multimodal Pass 1–5 in-repo; other citations remain [PROVISIONAL].
- **ANN nondeterminism:** recall is implementation-dependent; portable IR cannot claim bit-identical results.
- **Adapter impedance:** not all backends expose subgraph or hybrid fusion; shims may hide costs.
- **Optimism bias:** algebra elegance does not guarantee planner quality without measured cost models.

11 Conclusion

We framed RQL as a **portable retrieval-plan IR** for RAG—algebra plus physical planning under recall/latency/ACL—rather than as cosmetic SQL for vectors [HYPOTHESIS]. ACORN’s multimodal re-read [ESTABLISHED] the need to treat filter strategy as a first-class physical decision [Patel et al., 2024]. The immediate path is: finish Pass 1–5 on VBASE and Filtered-DiskANN, freeze a LogicalPlan schema, implement two adapters, and execute the evaluation plan with published artifacts—without shortcutting integrity.

A Reproducibility checklist

1. Scripts and configs under `experiments/` with pinned dependency files when benchmarks are added.
2. Dataset license notes and download digests (**HUMAN_TODO:** `experiments/protocols/02-datasets.r`).
3. Seeded RNGs where stochasticity exists; report ANN library versions in `ENV.txt`.
4. Store logical/physical plan JSON beside metrics (no orphaned scores).
5. Separate `AUTHOR_CLAIM` citations from `REPRODUCED` runs in result tables.
6. Research notes: journal + OKF-shaped bundles for paper reads.
7. Hand-off list for compute/credentials beyond the agent sandbox: `experiments/HUMAN_TODO.md`.

References

- Indri query language reference. <http://lemurproject.org/lemur/IndriQueryLanguage.php>, 2004. Classical IR language documentation; deep re-read pending.
- Substrait: Cross-language serialization for relational algebra. <https://substrait.io/>, 2024. Project site; accessed from research notes 2026-09-16.
- Open knowledge format (OKF). Google Cloud Platform, GitHub, 2025. URL <https://github.com/GoogleCloudPlatform/open-knowledge-format>. Also: <https://cloud.google.com/blog/products/data-analytics/how-the-open-knowledge-format-can-improve-data-sharing>.
- Sameer Agarwal et al. BlinkDB: Queries with bounded errors and bounded response times on very large data. In *EuroSys*, 2013. URL <https://arxiv.org/abs/1203.5485>. Author list abbreviated in draft BibTeX.

- Edmon Begoli, Jesús Camacho-Rodríguez, Julian Hyde, Michael J. Mior, and Daniel Lemire. Apache calcite: A foundational framework for optimized query processing over heterogeneous data sources. *arXiv preprint arXiv:1802.10233*, 2018. URL <https://arxiv.org/abs/1802.10233>.
- Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of SIGIR*, 2009. URL <https://cormack.uwaterloo.ca/cormacksigir09-rrf.pdf>.
- Laxman Dhulipala, Majid Hadian, Rajesh Jayaram, Jason Lee, and Vahab Mirrokni. MUVERA: Multi-vector retrieval via fixed dimensional encodings. *arXiv preprint arXiv:2405.19504*, 2024. URL <https://arxiv.org/abs/2405.19504>. Also NeurIPS 2024 per Google Research publication page; venue details verify before camera-ready.
- Jennie Duggan et al. The BigDAWG polystore system. *SIGMOD Record*, 2015. URL https://sigmod.org/publications/sigmodRecord/1506/pdfs/04_vision_Duggan.pdf. Author list abbreviated in draft BibTeX.
- Siddharth Gollapudi et al. Filtered-diskANN: Graph algorithms for approximate nearest neighbor search with filters. In *WWW*, 2023. URL <https://harsha-simhadri.org/pubs/Filtered-DiskANN23.pdf>. Provisional citation pending multimodal re-read; verify full author list.
- Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of SIGIR*, 2020. URL <https://arxiv.org/abs/2004.12832>.
- Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020. URL <https://arxiv.org/abs/2005.11401>. Author list abbreviated in draft BibTeX; verify full authors before camera-ready.
- Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2018. URL <https://arxiv.org/abs/1603.09320>.
- Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. Acorn: Performant and predicate-agnostic search over vector embeddings and structured data. *arXiv preprint arXiv:2403.04871*, 2024. URL <https://arxiv.org/abs/2403.04871>.
- Qianxi Zhang et al. VBASE: Unifying online vector similarity search and relational queries via relaxed monotonicity. In *OSDI*, 2023. URL https://www.usenix.org/system/files/osdi23-zhang-qianxi_1.pdf. Provisional citation; journal stub 0007 only — multimodal read pending.